Zapro Engineering Bootcamp

Incremental Project-Based Learning Guide

PART 1: Ruby Project — Library Management System

Days 1–10 | Build a CLI app from scratch, one concept per day

Day 1 — CLI Menu & Loops

Concepts

`puts` , `gets` , `chomp` , `loop do` , `while` , `for` , `times` , `each`

What to Build

Create `library.rb` . Show a menu, accept input, print placeholder messages. Nothing stored yet — just the skeleton running.

```
=== Library Management System ===
1. Add a book
2. List books
3. Search for a book
4. Update a book
5. Delete a book
6. Exit
Enter your choice:
```

Each option prints a placeholder like `"Add book – coming soon!"` . Option 6 exits.

Loop Formats — All of These Work

```ruby
# Option A: loop do (recommended — most common in Ruby)
loop do
  print "Enter choice: "
  choice = gets.chomp
  break if choice == "6"
```

```
  puts "You chose #{choice}"
end

# Option B: while
choice = ""
while choice != "6"
  print "Enter choice: "
  choice = gets.chomp
  puts "You chose #{choice}" unless choice == "6"
end

# Option C: for (rarely used, avoid in real code)
for i in 1..5
  puts "Item #{i}"
end

# Option D: times (for a fixed count)
3.times { |i| puts "Run #{i + 1}" }

# Option E: each (on an array — you'll use this a lot from Day 2)
[1, 2, 3].each { |n| puts n }
```

Use `loop do` for your menu. Add the others as comments so you understand them.

Expected Output

```
Enter your choice: 2
→ "List books - coming soon!"

Enter your choice: 6
→ "Goodbye!"
```

Git Checkpoint

```
git add library.rb
git commit -m "Day 1: CLI menu with loop and placeholder options"
```

Exercise — Do This On Your Own

Task: Add a hidden option `0` to the menu: **"Show menu in Spanish"**. When selected, re-print the entire menu but with Spanish labels (e.g. "Agregar un libro", "Listar libros", etc.). After showing the Spanish menu, ask the user to press Enter to return to the main menu.

Requirements:

- Spanish menu must show all 6 options
- Must return to the English menu loop after viewing

- Use a separate method `show_spanish_menu` (even though you haven't formally learned methods yet — try it)

What you'll practice: loop control, string output, breaking out of nested flow

```
git commit -m "Day 1 exercise: Spanish menu option"
```

Day 2 — Arrays & Hashes (State)

Concepts

Arrays `[]`, Hashes `{}`, `.push`, `.first`, `.last`, `.length`, `[]` access, array of hashes

What to Build

Replace the placeholder for "Add a book" with real logic. Store books as hashes in an array. List only the first 3.

```
books = []

books.push({ title: "Ruby on Rails", author: "DHH", year: 2005 })

books.first        # first book hash
books.last         # last book hash
books[0]           # index access
books.length       # how many books
```

Tasks

- `books` lives at the top of the file
- Add book: prompt for title, author, year — push a hash
- List books (option 2): show only first 3 using `books.first(3).each { |b| ... }`
- Add option 7: "List all books" — iterate all with `.each`
- Delete book: ask for title, remove using `.reject!`

Expected Output

```
Add a book
Title: Clean Code
Author: Robert Martin
Year: 2008
→ "Book added!"
```

```
List first 3 books:
1. Clean Code — Robert Martin (2008)
```

Git Checkpoint

```
git commit -m "Day 2: Store books as array of hashes, add/list/delete"
```

Exercise — Do This On Your Own

Task: Add menu option 8: **"Browse by genre"**. When adding a book, also ask for a genre (e.g. "Fiction", "Technical", "Biography"). Option 8 asks the user to type a genre and lists all matching books.

Requirements:

- Genre collected when adding a book
- Browsing is case-insensitive
- If no books match, print `"No books found in that genre."`
- If genre is left blank when adding, default to `"Uncategorized"`

What you'll practice: hash with more keys, array filtering, default values

```
git commit -m "Day 2 exercise: Browse books by genre"
```

Day 3 — Methods

Concepts

`def` , parameters, return values, method scope

What to Build

Your `library.rb` is getting long. Extract every action into its own method.

```
def show_menu
  puts "\n=== Library Management System ==="
  puts "1. Add a book"
  # ...
end

def add_book(books)
```

```
    print "Title: "
    title = gets.chomp
    books.push({ title: title, author: author, year: year.to_i })
    puts "Book added!"
  end

  def list_books(books)
    books.first(3).each_with_index do |book, i|
      puts "#{i + 1}. #{book[:title]} — #{book[:author]} (#{book[:year]})"
    end
  end
```

The loop becomes clean:

```
  loop do
    show_menu
    choice = gets.chomp
    case choice
    when "1" then add_book(books)
    when "2" then list_books(books)
    when "6" then puts "Goodbye!"; break
    end
  end
```

Tasks

- Extract: `show_menu` , `add_book(books)` , `list_books(books)` , `list_all_books(books)` , `delete_book(books)` , `search_book(books)`
- Each method does one thing only

Git Checkpoint

```
git commit -m "Day 3: Extract all actions into methods"
```

Exercise — Do This On Your Own

Task: Add a method `book_summary(books)` that prints a one-line report when the user picks option 9: **"Library summary"**. The summary should show: total books, the title of the most recently added book, and the oldest book by year.

Requirements:

- Must be a standalone method called `book_summary`
- Most recent = last item added
- Oldest = lowest year value

- Handle the case where the library is empty: print `"Library is empty."`

What you'll practice: methods with return values, array access patterns, edge case handling

```
git commit -m "Day 3 exercise: Library summary method"
```

Day 4 — Iteration & Enumeration

Concepts

`.each` , `.map` , `.select` , `.find` , `.reject` , `.count` , `.any?` , `.all?`

What to Build

Replace manual loops with Ruby's enumeration methods.

```ruby
def find_book(books, title)
  books.find { |b| b[:title] == title }
end

def books_by_author(books, author)
  books.select { |b| b[:author] == author }
end

def all_titles(books)
  books.map { |b| b[:title] }
end

books.count { |b| b[:year] > 2000 }
books.any?  { |b| b[:author] == "DHH" }
```

Tasks

- Update "Search" to use `.find`
- Update "Delete" to use `.reject!`
- Add option 8 (dev stats): total books, count after year 2000, all unique authors via `.map.uniq`

Git Checkpoint

```
git commit -m "Day 4: Use enumeration methods for search and filter"
```

Exercise — Do This On Your Own

Task: Add option 9: **"Books between years"**. The user enters a start year and end year, and the app lists all books published in that range, sorted by year (oldest first).

Requirements:

- Use `.select` to filter, then `.sort_by` to order
- If no books found in range, print an appropriate message
- If end year is less than start year, print `"Invalid range."` and return to menu

What you'll practice: chaining enumeration methods, range logic, input validation

```
git commit -m "Day 4 exercise: Filter books by year range"
```

Day 5 — Strings

Concepts

`.upcase` , `.downcase` , `.strip` , `.include?` , `.split` , `.gsub` , string interpolation, `.empty?`

What to Build

Make search case-insensitive. Validate that inputs aren't blank.

```ruby
books.find { |b| b[:title].downcase.include?(query.downcase) }

def validate_input(value, field_name)
  if value.strip.empty?
    puts "#{field_name} cannot be blank."
    return false
  end
  true
end

def display_book(book)
  puts "Title  : #{book[:title]}"
  puts "Author : #{book[:author]}"
  puts "Year   : #{book[:year]}"
  puts "-" * 30
end
```

Tasks

- Update `find_book` to be case-insensitive
- Validate all inputs with `.strip.empty?`

- Add `display_book(book)` and use it everywhere

Git Checkpoint

```
git commit -m "Day 5: Case-insensitive search and input validation"
```

Exercise — Do This On Your Own

Task: Add an **"Update book title"** option. The user types the current title to find the book, then types a new title. Before saving, show a preview: `"Rename 'Old Title' → 'New Title'? (y/n)"`. Only update if confirmed.

Requirements:

- Search must be case-insensitive
- New title must pass blank validation
- Trim leading/trailing spaces from the new title before saving
- Print `"No book found with that title."` if not found

What you'll practice: string manipulation, find + mutate pattern, confirmation flow

```
git commit -m "Day 5 exercise: Update book title with confirmation"
```

Day 6 — Classes & Objects

Concepts

`class`, `initialize`, `attr_accessor`, `attr_reader`, instance variables `@`, instance methods, `.new`

What to Build

Replace hashes with a `Book` class.

```ruby
class Book
  attr_accessor :title, :author, :year

  def initialize(title, author, year)
    @title  = title
    @author = author
    @year   = year.to_i
  end
```

```
  def display
    puts "Title  : #{@title}"
    puts "Author : #{@author}"
    puts "Year   : #{@year}"
    puts "-" * 30
  end

  def to_s
    "#{@title} by #{@author} (#{@year})"
  end
end
```

What Changes

```
# Before (hash)
books.push({ title: title, author: author, year: year })
puts book[:title]

# After (object)
books.push(Book.new(title, author, year))
puts book.title
```

Tasks

- Define `Book` class in the same file
- Replace hashes with `Book.new`
- Update all `book[:title]` references to `book.title`
- Call `book.display` everywhere

Git Checkpoint

```
git commit -m "Day 6: Introduce Book class, replace hashes with objects"
```

Exercise — Do This On Your Own

Task: Add a `age` method to the `Book` class that returns how many years old the book is (current year minus publication year). Add this to the display output: `"Age: 17 years"`. Also add a `recent?` method that returns true if the book was published in the last 5 years.

Requirements:

- `age` is an instance method on Book, not a standalone function
- `recent?` returns a boolean

- Update `display` to show age
- In the library summary (from Day 3 exercise), use `recent?` to count how many books are recent

What you'll practice: instance methods, computed values, method reuse

```
git commit -m "Day 6 exercise: Add age and recent? methods to Book"
```

Day 7 — Library Class

Concepts

Class managing a collection, multiple classes working together, `private` methods

What to Build

Create a `Library` class that owns `@books`.

```ruby
class Library
  def initialize
    @books = []
  end

  def add(title, author, year)
    @books.push(Book.new(title, author, year))
    puts "Book added!"
  end

  def list(limit: nil)
    collection = limit ? @books.first(limit) : @books
    collection.each_with_index { |book, i| puts "#{i + 1}. #{book}" }
  end

  def find(query)
    @books.find { |b| b.title.downcase.include?(query.downcase) }
  end

  def delete(title)
    before = @books.length
    @books.reject! { |b| b.title.downcase == title.downcase }
    @books.length < before ? puts("Deleted.") : puts("Not found.")
  end

  def size
    @books.length
  end
```

```
  private

  def find_index(title)
    @books.index { |b| b.title.downcase == title.downcase }
  end
end
```

Tasks

- Create `Library` class
- Instantiate once: `library = Library.new`
- Refactor the menu to call `library.add(...)`, `library.list`, etc.
- The `books` variable is gone — it lives in `@books` inside Library

Git Checkpoint

```
git commit -m "Day 7: Introduce Library class to manage book collection"
```

Exercise — Do This On Your Own

Task: Add a `stats` method to the `Library` class that returns a hash with: `total`, `by_genre` (a hash of genre → count), and `average_year`. Call it from option 9 in the menu and print each stat on its own line.

Requirements:

- `stats` must return a Hash, not print directly
- `by_genre` should look like `{ "Technical" => 3, "Fiction" => 2 }`
- `average_year` should be rounded to a whole number
- Handle empty library: all values should be 0 or empty hash

What you'll practice: methods that return data, hash building, arithmetic on object attributes

```
git commit -m "Day 7 exercise: Library stats method"
```

Day 8 — Modules & Mixins

Concepts

`module`, `include`, `extend`, reusable behavior, `Comparable`

What to Build

Extract `Displayable` and `Searchable` modules.

```ruby
module Displayable
  def display
    puts "Title  : #{title}"
    puts "Author : #{author}"
    puts "Year   : #{year}"
    puts "-" * 30
  end

  def to_s
    "#{title} — #{author} (#{year})"
  end
end

module Searchable
  def find_by_title(query)
    @books.find { |b| b.title.downcase.include?(query.downcase) }
  end

  def find_by_author(query)
    @books.select { |b| b.author.downcase.include?(query.downcase) }
  end
end

class Book
  include Displayable
  include Comparable

  attr_accessor :title, :author, :year

  def initialize(title, author, year)
    @title = title; @author = author; @year = year.to_i
  end

  def <=>(other)
    @year <=> other.year
  end
end

class Library
  include Searchable
end
```

Tasks

- Create `Displayable` module, include in `Book`
- Create `Searchable` module, include in `Library`

- Include `Comparable` in `Book`, define `<=>` by year
- Test: `library.books.sort` now works

Git Checkpoint

```
git commit -m "Day 8: Extract Displayable and Searchable modules, include Comparable"
```

Exercise — Do This On Your Own

Task: Create a `Exportable` module with a method `to_csv_row` that returns a comma-separated string of a book's fields: `"title,author,year,genre"`. Include it in `Book`. Add menu option 10: **"Export book to clipboard format"** — the user searches for a book by title and the app prints the CSV row for copy-pasting.

Requirements:

- `to_csv_row` must be defined in the `Exportable` module, not in `Book`
- If a field contains a comma, wrap it in double quotes
- If book not found, print appropriate message

What you'll practice: module design, string formatting, deciding what belongs in a module vs a class

```
git commit -m "Day 8 exercise: Exportable module with to_csv_row"
```

Day 9 — Inheritance & Error Handling

Concepts

Inheritance `<`, `super`, `begin/rescue/ensure`, `raise`, custom exception classes

What to Build

Add `DigitalBook` subclass. Add proper error handling.

```ruby
class BookNotFoundError < StandardError
  def initialize(title)
    super("Book not found: #{title}")
  end
end

class InvalidInputError < StandardError; end
```

```ruby
class DigitalBook < Book
  attr_accessor :url

  def initialize(title, author, year, url)
    super(title, author, year)
    @url = url
  end

  def display
    super
    puts "URL    : #{@url}"
    puts "-" * 30
  end
end
```

Wrap the menu loop:

```ruby
begin
  loop do
    # menu
  end
rescue BookNotFoundError => e
  puts "Error: #{e.message}"
rescue InvalidInputError => e
  puts "Invalid input: #{e.message}"
rescue Interrupt
  puts "\nGoodbye!"
ensure
  puts "Session ended."
end
```

Tasks

- Create `BookNotFoundError` and `InvalidInputError`
- Create `DigitalBook < Book` with `url`, override `display`
- Add option 9: "Add a digital book"
- Raise `InvalidInputError` if year is not a number

Git Checkpoint

```
git commit -m "Day 9: Add DigitalBook subclass and proper error handling"
```

Exercise — Do This On Your Own

Task: Create an `AudioBook < Book` subclass with a `duration_minutes` field (integer). Override `display` to show duration formatted as hours and minutes (e.g. `"Duration: 6h 32m"`). Add option 10: "Add an audiobook". Raise `InvalidInputError` if duration is not a positive integer.

Requirements:

- Duration must be a positive integer or raise error
- `display` calls `super` then adds the duration line
- The formatted output must show `Xh Ym` (e.g. `2h 5m`, `0h 45m`)
- All three book types (Book, DigitalBook, AudioBook) must coexist in the same `@books` array

What you'll practice: multiple inheritance levels, super chaining, input validation with custom errors

```
git commit -m "Day 9 exercise: AudioBook subclass with duration"
```

Day 10 — File I/O & Final Polish

Concepts

`File.read`, `File.write`, `require 'csv'`, CSV read/write, `at_exit`

What to Build

Persist the library to CSV so data survives restarts.

```ruby
require 'csv'

SAVE_FILE = "books.csv"

def save_library(library)
  CSV.open(SAVE_FILE, "w") do |csv|
    csv << ["title", "author", "year", "type", "url", "duration_minutes"]
    library.all_books.each do |book|
      if book.is_a?(DigitalBook)
        csv << [book.title, book.author, book.year, "digital", book.url, ""]
      elsif book.is_a?(AudioBook)
        csv << [book.title, book.author, book.year, "audio", "", book.duration_minutes
      else
        csv << [book.title, book.author, book.year, "physical", "", ""]
      end
    end
  end
  puts "Library saved."
end
```

```
def load_library(library)
  return unless File.exist?(SAVE_FILE)
  CSV.foreach(SAVE_FILE, headers: true) do |row|
    case row["type"]
    when "digital" then library.add_digital(row["title"], row["author"], row["year"].t
    when "audio"   then library.add_audio(row["title"], row["author"], row["year"].to_
    else                library.add(row["title"], row["author"], row["year"].to_i)
    end
  end
  puts "Loaded #{library.size} books from file."
end
```

Tasks

- Add `all_books` reader to Library
- Call `load_library` before the menu, `save_library` on exit
- Use `at_exit { save_library(library) }` as a safety net
- Final cleanup: no dead code, methods in the right class

Git Checkpoint

```
git add .
git commit -m "Day 10: Persist library to CSV — feature complete"
git tag v1.0
git push origin main --tags
```

Exercise — Do This On Your Own

Task: Add a **backup feature**. When the user exits (option 6), before saving `books.csv`, copy the previous `books.csv` to `books_backup.csv` (if it exists). This way they always have the last saved state.

Requirements:

- Use `File.exist?` to check if backup is needed
- Use `FileUtils.cp` (require 'fileutils') to copy the file
- Print `"Backup created: books_backup.csv"` when a backup is made
- Do not crash if no previous file exists

What you'll practice: File operations, require, defensive file handling

```
git commit -m "Day 10 exercise: Backup CSV before saving"
```

PART 2: Rails Project — Purchase Request Approval App

Days 11–30 | Build a full API-backed approval workflow, one layer per day

Day 11 — Rails Setup + Database & Redis Install

Concepts

`rails new`, MVC folder structure, `rails db:create`, `rails s`, Postgres, Redis

Postgres Setup

Mac:

```
brew install postgresql@14
brew services start postgresql@14
# Verify
psql postgres -c "SELECT version();"
```

Windows:

1. Download the installer from postgresql.org/download/windows
2. Run the installer — note the password you set for the `postgres` superuser
3. After install, open **SQL Shell (psql)** from the Start menu and verify it connects
4. Add Postgres to PATH (the installer usually does this): `C:\Program Files\PostgreSQL\14\bin`

```
# Verify in Command Prompt or PowerShell
psql -U postgres -c "SELECT version();"
```

For `config/database.yml` (both Mac and Windows):

```
default: &default
  adapter: postgresql
  encoding: unicode
  pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>
  username: <%= ENV.fetch("DB_USERNAME", "postgres") %>
  password: <%= ENV.fetch("DB_PASSWORD", "") %>
  host: localhost

development:
  <<: *default
```

```
    database: purchase_approval_app_development

test:
  <<: *default
  database: purchase_approval_app_test
```

> **Windows tip:** Set environment variables via System Properties → Advanced → Environment Variables, or use a `.env` file with the `dotenv-rails` gem.

Redis Setup

Redis is needed later for Sidekiq (background jobs).

Mac:

```
brew install redis
brew services start redis
# Verify
redis-cli ping   # → PONG
```

Windows: Redis does not have an official Windows build. Use one of these:

Option A — **WSL2** (recommended):

```
# Inside WSL2 terminal
sudo apt update && sudo apt install redis-server
sudo service redis-server start
redis-cli ping   # → PONG
```

Option B — **Docker**:

```
docker run -d -p 6379:6379 redis
# Verify
docker exec -it <container_id> redis-cli ping
```

Option C — **Memurai** (Windows-native Redis fork): Download from memurai.com, install, and it runs as a Windows service automatically.

Create the Rails App

```
rails new purchase_approval_app --database=postgresql --api
cd purchase_approval_app
rails db:create
rails s
```

Folder Structure

Folder	Purpose
`app/models/`	Database-backed classes
`app/controllers/`	Handle HTTP requests
`app/views/`	Response templates (Jbuilder)
`config/routes.rb`	URL → controller mapping
`db/migrate/`	Database change history
`Gemfile`	Dependencies

Git Checkpoint

```
git init && git add . && git commit -m "Day 11: Initialize Rails app with Postgres"
```

Exercise — Do This On Your Own

Task: The app will eventually support multiple environments. In `config/database.yml`, add a `staging` environment block that connects to a database named `purchase_approval_app_staging` and reads credentials from environment variables `STAGING_DB_USERNAME` and `STAGING_DB_PASSWORD`. Then write a brief comment in the file explaining why credentials should never be hardcoded.

What you'll practice: YAML config files, environment variables, Rails environment conventions

```
git commit -m "Day 11 exercise: Add staging database config"
```

Day 12 — First Model: User + Enums

Concepts

`rails generate model`, migrations, `schema.rb`, `rails db:migrate`, `rails console`, **enum**

What to Build

Generate the User model. Use a Rails enum for the `role` field instead of a plain string.

```
rails generate model User name:string email:string role:integer
rails db:migrate
```

Why Enum Instead of String?

```
# String — no safety, typos go unnoticed
user.role = "admine"   # silently wrong

# Enum — validated, readable, generates helper methods automatically
user.role = :admin     # raises if invalid
user.admin?            # → true
user.requester!        # sets and saves
User.approvers         # scope for free
```

The Model

```
class User < ApplicationRecord
  enum role: { requester: 0, approver: 1, admin: 2 }

  validates :name,  presence: true
  validates :email, presence: true, uniqueness: true
  validates :role,  presence: true
end
```

Console Exploration

```
alice = User.create!(name: "Alice Kumar", email: "alice@zapro.com", role: :requester)
bob   = User.create!(name: "Bob Singh",   email: "bob@zapro.com",   role: :approver)

alice.requester?    # → true
alice.approver?     # → false
bob.approver!       # sets to approver and saves

User.approvers      # all approvers (free scope from enum)
User.requesters     # all requesters
alice.role          # → "requester"
alice.role_before_type_cast  # → 0 (the integer stored in DB)
```

Tasks

- Generate User with `role:integer`
- Add enum in the model
- Create 3 users in console (one of each role)
- Test all enum helper methods: `?`, `!`, and the class-level scopes

Git Checkpoint

```
git commit -m "Day 12: Add User model with role enum"
```

Exercise — Do This On Your Own

Task: Add a `status` field to User using enum: `active: 0, inactive: 1, suspended: 2`. Write a migration to add the column (default: `active`). Then in the console, deactivate Alice (`alice.inactive!`), verify `alice.active?` returns false, and confirm `User.active` scope returns only active users.

What you'll practice: adding columns via migration, enum with defaults, combining two enums on one model

```
git commit -m "Day 12 exercise: Add user status enum"
```

Day 13 — PurchaseRequest Model + Callbacks + Byebug

Concepts

Second model, `belongs_to`, validations, **enum**, **model callbacks** (`before_validation`, `after_create`, `after_save`, `after_commit`), `byebug`

What to Build

Generate PurchaseRequest. Use enum for status. Add model lifecycle callbacks.

```
rails generate model PurchaseRequest title:string description:text amount:decimal stat
rails db:migrate
```

The Model

```
class PurchaseRequest < ApplicationRecord
  belongs_to :user

  enum status: { draft: 0, submitted: 1, approved: 2, rejected: 3 }

  validates :title,  presence: true
  validates :amount, presence: true, numericality: { greater_than: 0 }

  # Callbacks — run in this order on create:
  # before_validation → before_save → before_create → (INSERT) → after_create → after_
```

```ruby
  before_validation :set_default_status

  after_create  :log_creation
  after_save    :log_save
  after_commit  :log_commit, on: [:create, :update]

  private

  def set_default_status
    self.status ||= :draft
  end

  def log_creation
    Rails.logger.info "PurchaseRequest ##{id} created: #{title}"
  end

  def log_save
    Rails.logger.info "PurchaseRequest ##{id} saved (status: #{status})"
  end

  def log_commit
    # after_commit fires AFTER the database transaction is committed
    # Safe to do things like enqueue background jobs here
    Rails.logger.info "PurchaseRequest ##{id} committed to DB"
  end
end
```

Callback Order — Understand This

```
before_validation   → Good for setting defaults
after_validation    → Rarely used
before_save         → Runs on create AND update
before_create       → Runs only on create
[database INSERT]
after_create        → Runs only after create
after_save          → Runs on create AND update
after_commit        → Runs after transaction commits (safe for side effects)
```

Using Byebug

```ruby
def set_default_status
  byebug  # execution pauses here
  self.status ||= :draft
end
```

Run `PurchaseRequest.new` in console. Terminal pauses. Commands:

- Type any variable name to inspect it
- `next` to step to next line
- `continue` to resume
- `exit` to quit debugger

> Never commit `byebug` into code.

Tasks

- Generate PurchaseRequest with `status:integer`
- Add enum for status (draft, submitted, approved, rejected)
- Add callbacks: `before_validation` , `after_create` , `after_save` , `after_commit`
- Place `byebug` in `set_default_status` , trigger it, inspect `self` — then remove it
- Test in console: `PurchaseRequest.new.valid?` , `.errors.full_messages` , enum helpers

Git Checkpoint

```
git commit -m "Day 13: PurchaseRequest model with enum status and callbacks"
```

Exercise — Do This On Your Own

Task: Add an `after_save` callback called `notify_on_status_change` that prints to the Rails logger only when the status field changed (use `saved_change_to_status?`). The log message should include the old and new status values (use `saved_change_to_status` which returns `[old, new]`). Test it in the console by creating a PR and then manually changing its status.

What you'll practice: conditional callbacks, `saved_change_to_*` methods, understanding when after_save fires vs after_commit

```
git commit -m "Day 13 exercise: Log status changes in after_save callback"
```

Day 14 — Associations, Seeds & YML Config

Concepts

`has_many` , `belongs_to` , Faker gem, `seeds.rb` , **loading config from a YML file**

What to Build

Wire up associations. Seed data. Add `config/approval_settings.yml` and use it in the app.

Associations

```ruby
# app/models/user.rb
class User < ApplicationRecord
  has_many :purchase_requests, dependent: :destroy
end
```

YML Config File

```yaml
# config/approval_settings.yml
approval:
  auto_approve_below: 100
  max_amount: 50000
  default_currency: "USD"
  notify_approver_on_submit: true
  roles_that_can_approve:
    - approver
    - admin
```

Load it in an initializer:

```ruby
# config/initializers/approval_settings.rb
APPROVAL_SETTINGS = YAML.load_file(
  Rails.root.join("config", "approval_settings.yml")
).with_indifferent_access
```

Use it anywhere:

```ruby
APPROVAL_SETTINGS[:approval][:auto_approve_below]   # → 100
APPROVAL_SETTINGS[:approval][:max_amount]            # → 50000
APPROVAL_SETTINGS.dig(:approval, :roles_that_can_approve)  # → ["approver", "admin"]
```

Seed File

```ruby
# db/seeds.rb
require 'faker'

User.destroy_all

alice = User.create!(name: "Alice Kumar", email: "alice@zapro.com", role: :requester)
bob   = User.create!(name: "Bob Singh",   email: "bob@zapro.com",   role: :approver)
carol = User.create!(name: "Carol Patel", email: "carol@zapro.com", role: :admin)

10.times do
  alice.purchase_requests.create!(
```

```
      title:       Faker::Commerce.product_name,
      description: Faker::Lorem.sentence,
      amount:      Faker::Commerce.price(range: 100..10000)
    )
  end

  puts "Seeded #{User.count} users, #{PurchaseRequest.count} purchase requests"
```

```
  bundle add faker
  rails db:seed
```

Console Exploration

```
  alice = User.find_by(name: "Alice Kumar")
  alice.purchase_requests
  alice.purchase_requests.count
  pr = alice.purchase_requests.first
  pr.user.name

  # Use the YML config
  APPROVAL_SETTINGS.dig(:approval, :auto_approve_below)
```

Tasks

- Add `has_many :purchase_requests` to User
- Create `config/approval_settings.yml` with the structure above
- Create `config/initializers/approval_settings.rb` to load it
- Write seeds.rb using Faker, run `rails db:seed`
- Confirm `APPROVAL_SETTINGS` is accessible in console

Git Checkpoint

```
  git commit -m "Day 14: Associations, seeds, and approval_settings.yml"
```

Exercise — Do This On Your Own

Task: Add a `max_title_length` key to `approval_settings.yml` (set it to 100). Add a validation to the `PurchaseRequest` model that uses `APPROVAL_SETTINGS.dig(:approval, :max_title_length)` as the max length constraint instead of a hardcoded number. Write a comment explaining why reading from config is better than hardcoding.

What you'll practice: using config in validations, YAML structure, Rails initializers

```
git commit -m "Day 14 exercise: Validate title length from YML config"
```

Day 15 — Routes, Controllers & Controller Callbacks

Concepts

`resources`, `rails routes`, controller actions, `render json:`, `params`, **controller callbacks** (`before_action`, `after_action`, `around_action`)

What to Build

Add routes, controller, and controller-level callbacks.

```
# config/routes.rb
Rails.application.routes.draw do
  namespace :api do
    resources :purchase_requests, only: [:index, :show, :create, :update] do
      member do
        post :submit
        post :approve
        post :reject
      end
    end
  end
end
```

Controller Callbacks

```
class Api::PurchaseRequestsController < ApplicationController
  # before_action: runs before specified actions
  before_action :set_purchase_request, only: [:show, :update, :submit, :approve, :reje
  before_action :set_current_user

  # after_action: runs after the action, response already built
  after_action :log_response

  # around_action: wraps the action — you control when it runs
  around_action :measure_time, only: [:index]

  def index
    @purchase_requests = PurchaseRequest.includes(:user).all
    render json: { status: "success", data: { purchase_requests: @purchase_requests }
  end

  def show
```

```
    # @purchase_request is the full ActiveRecord object — all columns from the DB row.
    # Rendering it directly with `render json: @purchase_request` would dump every col
    # including internal ones (updated_at, user_id foreign key, etc.) that the fronten
    # doesn't need. Instead, pick only what you want to expose:
    render json: {
      status: "success",
      data: {
        purchase_request: {
          id:          @purchase_request.id,
          title:       @purchase_request.title,
          description: @purchase_request.description,
          amount:      @purchase_request.amount,
          status:      @purchase_request.status,
          created_at:  @purchase_request.created_at,
          user: {
            id:    @purchase_request.user.id,
            name:  @purchase_request.user.name,
            email: @purchase_request.user.email
          }
        }
      }
    }
    # Note: From Day 17 onward we'll use Jbuilder templates to do this more cleanly.
    # The pattern is the same — you decide which fields to include, not Rails.
  end

  def create
    @purchase_request = PurchaseRequest.new(purchase_request_params)
    if @purchase_request.save
      render json: { status: "success", data: { purchase_request: @purchase_request }
    else
      render json: { status: "failure", message: @purchase_request.errors.full_message
    end
  end

  private

  def set_purchase_request
    @purchase_request = PurchaseRequest.find(params[:id])
  end

  def set_current_user
    # Placeholder — Devise will handle this properly on Day 16
    @current_user = User.find_by(id: params[:user_id]) || User.first
  end

  def log_response
    Rails.logger.info "#{request.method} #{request.path} → #{response.status}"
  end

  def measure_time
    start = Time.current
    yield   # runs the action
```

```
      elapsed = Time.current - start
      Rails.logger.info "index took #{elapsed.round(3)}s"
    end

    def purchase_request_params
      params.require(:purchase_request).permit(:title, :description, :amount, :user_id)
    end
  end
```

Callback Order

```
before_action  → before the action runs (auth checks, loading records)
around_action  → yield wraps the action (timing, transactions)
[action runs]
after_action   → after response is built (logging, headers)
```

Tasks

- Create `Api::PurchaseRequestsController` with `before_action :set_purchase_request`
- Add `after_action` for logging
- Add `around_action` for timing on index
- Run `rails routes | grep purchase` to see all routes
- Test with curl: GET index, GET show, POST create

Git Checkpoint

```
git commit -m "Day 15: Routes and controller with before/after/around_action"
```

Exercise — Do This On Your Own

Task: Add a `before_action :validate_amount_limit` that runs only on `create`. It should check if `params[:purchase_request][:amount].to_f` exceeds `APPROVAL_SETTINGS.dig(:approval, :max_amount)`. If it does, render a failure response with message `"Amount exceeds the maximum allowed"` and halt the action (use `return` after render). Test with a curl request that passes an amount over the limit.

What you'll practice: conditional before_action, halting the filter chain, using YML config in controllers

```
git commit -m "Day 15 exercise: Validate amount limit in before_action"
```

Day 16 — Devise Authentication

Concepts

Devise gem, `current_user`, session cookies vs JWT, `authenticate_user!`, Devise helpers

What to Build

Add Devise to handle user authentication. Understand how cookies work in the process.

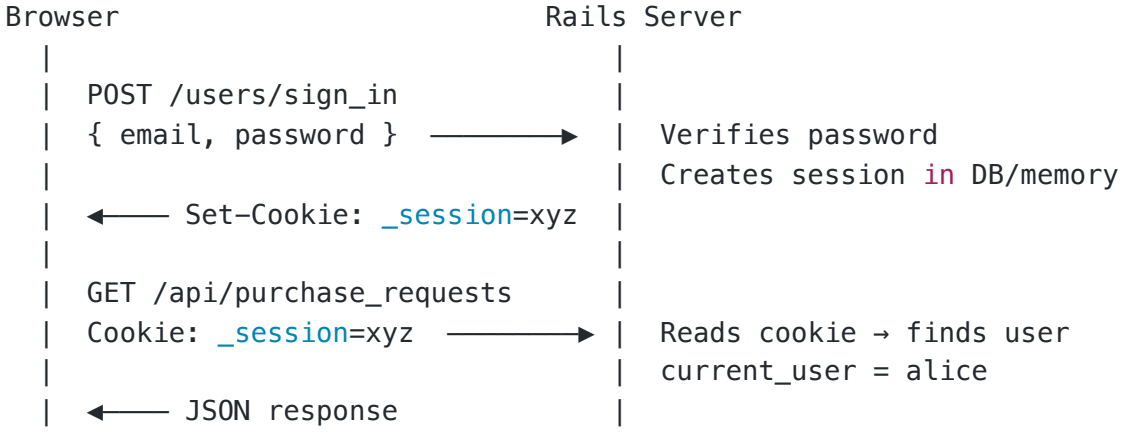
```
bundle add devise
rails generate devise:install
rails generate devise User
rails db:migrate
```

> **Note:** If User model already exists, Devise will add columns to it (confirmation tokens, encrypted password, etc.). Check the generated migration carefully before running.

How Authentication Works (Cookies)

When a user logs in:

1. Browser sends `POST /users/sign_in` with email + password
2. Rails verifies credentials, creates a **session**
3. Rails sends back a **cookie** (`_session_id`) containing an encrypted session identifier
4. Browser stores this cookie automatically
5. Every subsequent request sends the cookie back
6. Rails reads the cookie → looks up the session → knows who `current_user` is



Configure Devise

```
# config/initializers/devise.rb
Devise.setup do |config|
  config.mailer_sender = 'noreply@zapro.com'
  # For API-only apps, use token auth or sessions with cookie_store
end



# app/controllers/application_controller.rb
class ApplicationController < ActionController::API
  include ActionController::Cookies  # needed for API mode

  before_action :authenticate_user!

  def current_user
    # Devise provides this automatically
    super
  end
end
```

Why ApplicationController Matters

Every controller in your app inherits from `ApplicationController`:

```
ActionController::API          ← Rails base (HTTP handling)
        ↑
ApplicationController          ← YOUR base (auth, shared behavior)
        ↑                ↑
PurchaseRequestsController   UsersController  (and every other controller)
```

This means anything you put in `ApplicationController` automatically applies to **all controllers** — you write it once and every endpoint gets it. This is exactly where cross-cutting concerns belong:

- `before_action :authenticate_user!` — every endpoint requires login, without repeating it in each controller
- `current_user` — available in every controller and every Jbuilder view
- Common error handling, logging, header setting

If you only want auth on *some* controllers, you can skip it selectively:

```
class Api::PublicController < ApplicationController
  skip_before_action :authenticate_user!  # this controller is public
end
```

Update the Controller

Now replace the placeholder `set_current_user` with real Devise auth:

```
# Remove the old set_current_user before_action
# Devise's authenticate_user! in ApplicationController handles it

def create
  pr = PurchaseRequest.new(purchase_request_params.merge(user: current_user))
  # ...
end
```

Tasks

- Install Devise, run `rails generate devise User`, run migration
- Add `authenticate_user!` to ApplicationController
- Test: hitting any endpoint without auth should return 401
- Sign in via `POST /users/sign_in` with email/password, check the cookie in the response headers
- Use `current_user` in controller instead of the placeholder

Git Checkpoint

```
git commit -m "Day 16: Add Devise authentication with current_user"
```

Exercise — Do This On Your Own

Task: Add a `GET /api/me` endpoint in a new `Api::UsersController` that returns the current user's name, email, and role as JSON. It should only work if the user is authenticated (return failure response if not). Test both authenticated and unauthenticated access.

What you'll practice: Devise's `current_user`, authentication guard, simple JSON response

```
git commit -m "Day 16 exercise: Add /api/me endpoint"
```

Day 17 — Jbuilder

Concepts

Jbuilder partials, `json.extract!`, `json.partial!`, reusable JSON templates

What to Build

Replace `render json:` with structured Jbuilder templates.

Add `gem 'jbuilder'` if not present.

```
app/views/api/purchase_requests/
├── index.json.jbuilder
├── show.json.jbuilder
└── _purchase_request.json.jbuilder
```

```
# _purchase_request.json.jbuilder
json.extract! purchase_request, :id, :title, :description, :amount, :status, :created_

json.user do
  json.extract! purchase_request.user, :id, :name, :email, :role
end
```

```
# index.json.jbuilder
json.status "success"
json.data do
  json.purchase_requests @purchase_requests,
    partial: "api/purchase_requests/purchase_request",
    as: :purchase_request
end
```

```
# show.json.jbuilder
json.status "success"
json.data do
  json.partial! "api/purchase_requests/purchase_request",
    purchase_request: @purchase_request
end
```

Tasks

- Create the partial with `json.extract!`
- Create index and show templates
- Update controller to use `@instance_variables`
- Verify JSON output: only the fields you want, no noise

Git Checkpoint

```
git commit -m "Day 17: Add Jbuilder templates for JSON responses"
```

Exercise — Do This On Your Own

Task: Create a `GET /api/purchase_requests?status=submitted` filter. When a `status` param is present, only return PRs with that status. Update the Jbuilder index template to also include a `meta` block at the top level with `total_count` and `filtered_by` (the status param value, or `"all"` if none).

What you'll practice: query params in controller, conditional scoping, adding metadata to Jbuilder response

```
git commit -m "Day 17 exercise: Status filter with meta in Jbuilder"
```

Day 18 — Concerns & Helpers

Concepts

`ActiveSupport::Concern`, `included`, `module_function`, controller concerns, model concerns, helper modules

What to Build

Extract reusable logic into concerns. Add a helper for formatting amounts.

Model Concern

```
# app/models/concerns/status_trackable.rb
module StatusTrackable
  extend ActiveSupport::Concern

  included do
    after_save :track_status_change, if: :saved_change_to_status?
  end

  def status_changed_to?(new_status)
    saved_change_to_status&.last == new_status.to_s
  end

  private

  def track_status_change
    old_status, new_status = saved_change_to_status
    Rails.logger.info "#{self.class.name} ##{id}: #{old_status} → #{new_status}"
  end
end
```

Include it in PurchaseRequest:

```ruby
class PurchaseRequest < ApplicationRecord
  include StatusTrackable
  # ...
end
```

Controller Concern

```ruby
# app/controllers/concerns/pagination.rb
module Pagination
  extend ActiveSupport::Concern

  def paginate(scope)
    page     = (params[:page] || 1).to_i
    per_page = (params[:per_page] || 20).to_i
    scope.limit(per_page).offset((page - 1) * per_page)
  end

  def pagination_meta(scope)
    {
      total:    scope.count,
      page:     (params[:page] || 1).to_i,
      per_page: (params[:per_page] || 20).to_i
    }
  end
end



class Api::PurchaseRequestsController < ApplicationController
  include Pagination

  def index
    scope = PurchaseRequest.includes(:user).all
    @purchase_requests = paginate(scope)
    @meta = pagination_meta(scope)
  end
end
```

Helper Module

```ruby
# app/helpers/amount_helper.rb
module AmountHelper
  def format_amount(amount, currency = "USD")
    "$#{format('%.2f', amount)}"
  end

  def humanize_status(status)
    status.to_s.gsub("_", " ").capitalize
```

```
    end
  end
```

Use in Jbuilder:

```
# In a jbuilder file
json.formatted_amount format_amount(purchase_request.amount)
json.status_label     humanize_status(purchase_request.status)
```

Tasks

- Create `StatusTrackable` concern, include in PurchaseRequest
- Create `Pagination` concern, include in controller
- Create `AmountHelper`, use in Jbuilder
- Test pagination: `GET /api/purchase_requests?page=1&per_page=3`

Git Checkpoint

```
git commit -m "Day 18: Add StatusTrackable concern, Pagination concern, AmountHelper"
```

Exercise — Do This On Your Own

Task: Create a `Timestampable` model concern that adds two methods: `created_ago` (returns a human-readable string like `"3 days ago"` using `time_ago_in_words`) and `formatted_created_at` (returns date as `"Jun 04, 2026"`). Include it in PurchaseRequest and expose both fields in the Jbuilder partial.

What you'll practice: concern design, Rails time helpers, when to put logic in concerns vs models

```
git commit -m "Day 18 exercise: Timestampable concern with time formatting"
```

Day 19 — Scopes & ActiveRecord Queries

Concepts

`scope`, `where`, `includes`, `joins`, `order`, N+1 queries

What to Build

Add named scopes to PurchaseRequest. Fix N+1.

```ruby
class PurchaseRequest < ApplicationRecord
  scope :recent,   -> { order(created_at: :desc) }
  scope :by_user,  ->(user_id) { where(user_id: user_id) }
  scope :above,    ->(amount)  { where("amount > ?", amount) }
  scope :for_role, ->(role)    { joins(:user).where(users: { role: User.roles[role] }) }
end
```

N+1 Demo

```ruby
# BAD — 1 query per row
PurchaseRequest.all.each { |pr| puts pr.user.name }

# GOOD — 2 queries total
PurchaseRequest.includes(:user).all.each { |pr| puts pr.user.name }

# joins — filter by association columns
PurchaseRequest.joins(:user).where(users: { role: User.roles[:approver] })
```

Tasks

- Add scopes: `recent`, `by_user`, `above`, `for_role`
- Demo N+1 in console, fix with `includes`
- Update controller index to use scopes and `includes`

Git Checkpoint

```
git commit -m "Day 19: Add scopes and fix N+1 with includes"
```

Exercise — Do This On Your Own

Task: Add a `GET /api/purchase_requests/summary` route (collection action). It should return a JSON object where each key is a status name and the value is the count and total amount for that status. Example: `{ "draft": { count: 3, total_amount: "1500.00" }, "submitted": { ... } }`. Use `group` and `sum` in ActiveRecord, not Ruby iteration.

What you'll practice: ActiveRecord aggregation (group, sum, count), collection routes, avoiding N+1 with aggregates

```
git commit -m "Day 19 exercise: Summary endpoint with grouped aggregates"
```

Day 20 — AASM State Machine

Concepts

AASM gem, states, events, transitions, guards, callbacks on transitions

Why State Machine?

Right now status is just an integer we set manually. A state machine enforces valid transitions — you can't approve a PR that hasn't been submitted first.

```
bundle add aasm
```

```
class PurchaseRequest < ApplicationRecord
  include AASM

  enum status: { draft: 0, submitted: 1, approved: 2, rejected: 3 }

  aasm column: :status, enum: true do
    state :draft, initial: true
    state :submitted
    state :approved
    state :rejected

    event :submit do
      transitions from: :draft, to: :submitted
      after do
        NotifyApproverJob.perform_later(id)
      end
    end

    event :approve do
      transitions from: :submitted, to: :approved, guard: :approver_present?
      after do |approver|
        ApprovalEvent.create!(purchase_request: self, user: approver, action: "approve
      end
    end

    event :reject do
      transitions from: :submitted, to: :rejected
      after do |approver, reason|
        ApprovalEvent.create!(
          purchase_request: self,
          user:             approver,
          action:           "rejected",
          comment:          reason
        )
      end
```

```
      end
    end

    def approver_present?
      user.approver? || user.admin?
    end
  end
```

Using It

```
pr = PurchaseRequest.find(1)

pr.may_submit?    # → true (can we call submit? event)
pr.submit!        # transitions to :submitted, fires after callback
pr.may_approve?   # → true
pr.approve!(approver_user)
pr.approved?      # → true

# Invalid transition raises AASM::InvalidTransition
pr.submit!        # → raises (already submitted)
```

Update Services

Now services call AASM events instead of manually setting status:

```
# In SubmitService
def perform
  validate!
  @purchase_request.submit!
  true
rescue AASM::InvalidTransition => e
  @purchase_request.errors.add(:base, "Cannot submit: #{e.message}")
  false
end
```

Tasks

- Add AASM gem
- Define states and events in PurchaseRequest
- Update SubmitService, ApproveService, RejectService to use `submit!`, `approve!`, `reject!`
- Test in console: valid transitions, invalid transitions, guard checks
- Try `may_submit?`, `may_approve?` helpers

Git Checkpoint

```
git commit -m "Day 20: Add AASM state machine for PurchaseRequest transitions"
```

Exercise — Do This On Your Own

Task: Add a `cancel` event to the state machine that allows transitioning from `draft` or `submitted` → `cancelled` (add `cancelled: 4` to the enum). The guard for cancelling a submitted PR is that `current_user` must be the original requester. Add a `cancelled?` helper and make sure the cancel action is reflected in ApprovalEvent when a submitted PR is cancelled.

What you'll practice: multiple source states in a transition, guard logic, adding to an existing state machine

```
git commit -m "Day 20 exercise: Add cancel event to state machine"
```

Day 21 — Raw SQL & PostgreSQL Indexing

Concepts

Raw SQL in Rails, `SELECT`, `JOIN`, `GROUP BY`, database indexes

Raw SQL Practice

Create `db/sql_practice.sql`:

```sql
-- Count PRs by status
SELECT status, COUNT(*) as total, ROUND(SUM(amount)::numeric, 2) as total_amount
FROM purchase_requests
GROUP BY status;

-- Requesters with their total spend
SELECT u.name, u.email, COUNT(pr.id) as pr_count, SUM(pr.amount) as total_spend
FROM users u
JOIN purchase_requests pr ON pr.user_id = u.id
GROUP BY u.id, u.name, u.email
ORDER BY total_spend DESC;

-- Approved PRs above $500
SELECT pr.title, pr.amount, u.name as requester
FROM purchase_requests pr
JOIN users u ON u.id = pr.user_id
WHERE pr.status = 2 AND pr.amount > 500;
```

In Rails console:

```
result = ActiveRecord::Base.connection.execute(
  "SELECT status, COUNT(*) as total FROM purchase_requests GROUP BY status"
)
result.each { |row| puts row.inspect }
```

Add Indexes

```
rails generate migration AddIndexesToPurchaseRequests
```

```
def change
  add_index :purchase_requests, :status
  add_index :purchase_requests, [:user_id, :status]
end
```

Tasks

- Write 3 SQL queries in `db/sql_practice.sql`
- Run one from Rails console
- Add indexes migration
- Check `db/schema.rb` for index listings

Git Checkpoint

```
git commit -m "Day 21: SQL practice queries and database indexes"
```

Exercise — Do This On Your Own

Task: Write a raw SQL query (in `db/sql_practice.sql`) that finds the **top 3 requesters by total approved spend** — showing their name, number of approved PRs, and total approved amount. Then expose this as a `GET /api/reports/top_requesters` endpoint that runs this query and returns the result as JSON. Use `ActiveRecord::Base.connection.execute` or `.select` with the raw SQL.

What you'll practice: raw SQL with aggregation, Rails reporting endpoints, returning raw SQL results as JSON

```
git commit -m "Day 21 exercise: Top requesters report endpoint"
```

Day 22 — Service Objects

Concepts

Service object pattern, single public `perform` method, private methods

What to Build

All business logic goes in services. Controllers stay thin.

```
app/services/purchase_requests/
├── submit_service.rb
├── approve_service.rb
└── reject_service.rb
```

```ruby
# app/services/purchase_requests/submit_service.rb
module PurchaseRequests
  class SubmitService
    def initialize(purchase_request, current_user)
      @purchase_request = purchase_request
      @current_user     = current_user
    end

    def perform
      validate_ownership!
      @purchase_request.submit!   # AASM event
      true
    rescue AASM::InvalidTransition, ActiveRecord::RecordInvalid => e
      @purchase_request.errors.add(:base, e.message)
      false
    end

    private

    def validate_ownership!
      unless @purchase_request.user == @current_user
        raise ActiveRecord::RecordInvalid, "You can only submit your own requests"
      end
    end
  end
end
```

Controller becomes thin:

```ruby
def submit
  service = PurchaseRequests::SubmitService.new(@purchase_request, current_user)
  if service.perform
```

```
      render json: { status: "success", data: { purchase_request: @purchase_request.relo
    else
      render json: { status: "failure", message: @purchase_request.errors.full_messages.
    end
  end
```

Tasks

- Create SubmitService, ApproveService, RejectService
- Each uses AASM events internally
- Controller actions call services and render results
- Test submit → approve flow with curl

Git Checkpoint

```
git commit -m "Day 22: Service objects for submit, approve, reject"
```

Exercise — Do This On Your Own

Task: Create `PurchaseRequests::BulkApproveService` that accepts an array of PR IDs and an approver user. It should attempt to approve each one, collecting results into a hash: `{ approved: [ids], failed: [{ id:, reason: }] }`. Return this structure from `perform`. Add a `POST /api/purchase_requests/bulk_approve` endpoint that calls it.

What you'll practice: service objects with complex return values, collection operations, partial success handling

```
git commit -m "Day 22 exercise: BulkApproveService and endpoint"
```

Day 23 — RSpec Setup & Model Specs

Concepts

RSpec, `describe`, `context`, `it`, `expect`, FactoryBot, `let`, `before`

```
group :test do
  gem 'rspec-rails'
  gem 'factory_bot_rails'
end
```

```
bundle install
rails generate rspec:install
```

Factories

```
# spec/factories/users.rb
FactoryBot.define do
  factory :user do
    name  { Faker::Name.name }
    email { Faker::Internet.unique.email }
    role  { :requester }

    trait :approver do
      role { :approver }
    end

    trait :admin do
      role { :admin }
    end
  end
end
```

```
# spec/factories/purchase_requests.rb
FactoryBot.define do
  factory :purchase_request do
    title       { Faker::Commerce.product_name }
    description { Faker::Lorem.sentence }
    amount      { 2000.0 }
    association :user

    trait :submitted do
      status { :submitted }
    end

    trait :approved do
      status { :approved }
    end
  end
end
```

Model Spec

```
RSpec.describe PurchaseRequest, type: :model do
  describe "validations" do
    it "is valid with all required fields" do
      expect(build(:purchase_request)).to be_valid
    end
```

```
    it "is invalid without a title" do
      pr = build(:purchase_request, title: nil)
      expect(pr).not_to be_valid
      expect(pr.errors[:title]).to include("can't be blank")
    end

    it "is invalid with a negative amount" do
      expect(build(:purchase_request, amount: -50)).not_to be_valid
    end
  end

  describe "scopes" do
    let!(:draft_pr)    { create(:purchase_request) }
    let!(:approved_pr) { create(:purchase_request, :approved) }

    it "returns only draft requests" do
      expect(PurchaseRequest.draft).to include(draft_pr)
      expect(PurchaseRequest.draft).not_to include(approved_pr)
    end
  end

  describe "AASM transitions" do
    let(:pr) { create(:purchase_request) }

    it "transitions from draft to submitted" do
      pr.submit!
      expect(pr.reload.status).to eq("submitted")
    end

    it "cannot approve a draft PR" do
      expect { pr.approve! }.to raise_error(AASM::InvalidTransition)
    end
  end
end
```

Tasks

- Set up RSpec and FactoryBot
- Create factories for User and PurchaseRequest with traits
- Write model spec: validations, scopes, AASM transitions
- Run `rspec spec/models/`

Git Checkpoint

```
git commit -m "Day 23: RSpec setup, factories, model specs"
```

Exercise — Do This On Your Own

Task: Write model specs for the `User` model covering: (1) presence validation on name and email, (2) uniqueness validation on email — use two `create(:user)` calls to test uniqueness, (3) enum behavior — verify `user.approver?` returns false for a requester, (4) the `active` scope from Day 12 exercise. All specs must pass.

What you'll practice: writing specs for enums, uniqueness validations, scope specs

```
git commit -m "Day 23 exercise: User model specs"
```

Day 24 — Request Specs

Concepts

Request specs, `post`, `get`, JSON parsing, end-to-end API testing

```
# spec/requests/api/purchase_requests_spec.rb
RSpec.describe "Api::PurchaseRequests", type: :request do
  let(:requester) { create(:user) }
  let(:approver)  { create(:user, :approver) }

  describe "GET /api/purchase_requests" do
    before { create_list(:purchase_request, 3, user: requester) }

    it "returns all purchase requests" do
      sign_in requester  # Devise test helper
      get "/api/purchase_requests"
      json = JSON.parse(response.body)

      expect(response).to have_http_status(:ok)
      expect(json["status"]).to eq("success")
      expect(json["data"]["purchase_requests"].count).to eq(3)
    end
  end

  describe "POST /api/purchase_requests/:id/submit" do
    let(:pr) { create(:purchase_request, user: requester) }

    it "submits a draft PR" do
      sign_in requester
      post "/api/purchase_requests/#{pr.id}/submit", as: :json
      expect(pr.reload.status).to eq("submitted")
    end

    it "fails when user does not own the PR" do
      other_user = create(:user)
      sign_in other_user
      post "/api/purchase_requests/#{pr.id}/submit", as: :json
```

```
        json = JSON.parse(response.body)
        expect(json["status"]).to eq("failure")
      end
    end
  end
```

Tasks

- Write request specs for index, create, submit, approve
- Each spec checks HTTP status, JSON shape, and DB state
- Run `rspec spec/requests/`

Git Checkpoint

```
git commit -m "Day 24: Request specs for purchase request API"
```

Exercise — Do This On Your Own

Task: Write a request spec for the `bulk_approve` endpoint (Day 22 exercise). Test three scenarios: (1) all PRs approved successfully, (2) mixed result — some approved, some fail (wrong status), (3) unauthorized user tries to bulk approve. Each spec must assert the JSON response structure and the actual database state.

What you'll practice: complex request specs, asserting partial success, multiple assertion patterns

```
git commit -m "Day 24 exercise: Request specs for bulk_approve"
```

Day 25 — ActiveJob & Sidekiq

Concepts

`ActiveJob` , `perform_later` , Sidekiq, Redis, background workers

```
bundle add sidekiq
```

```
# config/application.rb
config.active_job.queue_adapter = :sidekiq
```

```
# app/jobs/notify_approver_job.rb
class NotifyApproverJob < ApplicationJob
  queue_as :default

  def perform(purchase_request_id)
    pr       = PurchaseRequest.find(purchase_request_id)
    approver = User.approvers.first
    return unless approver

    Rails.logger.info "Notifying #{approver.name} about: #{pr.title} (#{pr.amount})"
    # In a real app: ActionMailer or push notification goes here
  end
end
```

The AASM `submit` event already calls `NotifyApproverJob.perform_later(id)`.

Start Sidekiq:

```
redis-server &
bundle exec sidekiq
```

Job Spec

```
RSpec.describe NotifyApproverJob, type: :job do
  it "is enqueued when a PR is submitted" do
    pr = create(:purchase_request)
    expect {
      PurchaseRequests::SubmitService.new(pr, pr.user).perform
    }.to have_enqueued_job(NotifyApproverJob).with(pr.id)
  end
end
```

Tasks

- Add Sidekiq, configure as queue adapter
- Start Redis + Sidekiq, submit a PR, watch the logs
- Write job spec

Git Checkpoint

```
git commit -m "Day 25: NotifyApproverJob with Sidekiq"
```

Exercise — Do This On Your Own

Task: Create `WeeklyReportJob` that runs once (manually for now) and logs a summary: total PRs created this week by status, average amount, and the name of the approver who approved the most PRs. Schedule it with `WeeklyReportJob.perform_later`. Write a job spec that verifies the job can be enqueued and that it does not raise when performed (use `perform_now` in the spec).

What you'll practice: job design, querying inside jobs, job specs with perform_now

```
git commit -m "Day 25 exercise: WeeklyReportJob"
```

Day 26 — Data Migrations

Concepts

Additive migrations, backfill patterns, safe batched updates

What to Build

Add `submitted_at` to PurchaseRequests. Backfill it for existing rows.

```
rails generate migration AddSubmittedAtToPurchaseRequests submitted_at:datetime
rails generate migration BackfillSubmittedAtForPurchaseRequests
```

Simple Batched Backfill (No CTE Needed)

```ruby
class BackfillSubmittedAtForPurchaseRequests < ActiveRecord::Migration[7.0]
  def up
    # Find and update in batches — simple and readable
    PurchaseRequest.where(status: 1, submitted_at: nil)
                   .in_batches(of: 1000) do |batch|
      batch.update_all("submitted_at = updated_at")
    end
  end

  def down
    # Not reversible
  end
end
```

> `in_batches` is built into Rails — no raw SQL needed. It handles the batching loop for you, preventing long-running transactions on large tables.

Update the AASM `submit` event to set `submitted_at`:

```
  event :submit do
    transitions from: :draft, to: :submitted
    after { update_columns(submitted_at: Time.current) }
  end
```

Tasks

- Write both migrations
- Run them: `rails db:migrate`
- Verify: `PurchaseRequest.submitted.where.not(submitted_at: nil).count`
- Update AASM submit event to populate `submitted_at`

Git Checkpoint

```
git commit -m "Day 26: Add submitted_at column with batched backfill"
```

Exercise — Do This On Your Own

Task: Add an `approved_at:datetime` column. Write a backfill migration for existing approved PRs. Then update the AASM `approve` event to set `approved_at` . Add a scope `approved_this_month` that filters by `approved_at` in the current calendar month. Test the scope in console after seeding some approved PRs.

What you'll practice: migration patterns, scope with date conditions, keeping AASM events in sync with timestamps

```
git commit -m "Day 26 exercise: approved_at column and approved_this_month scope"
```

Day 27 — Approval History Model

Concepts

New audit model, `has_many` , audit trail pattern

```
rails generate model ApprovalEvent purchase_request:references user:references action:
rails db:migrate
```

```
class ApprovalEvent < ApplicationRecord
  belongs_to :purchase_request
```

```
  belongs_to :user

  validates :action, inclusion: { in: %w[submitted approved rejected cancelled] }
  before_validation { self.occurred_at ||= Time.current }
end

# purchase_request.rb
has_many :approval_events, dependent: :destroy
```

Update services (or AASM after-callbacks) to create events.

Update Jbuilder show:

```
json.approval_history @purchase_request.approval_events.order(:occurred_at) do |event|
  json.action  event.action
  json.by      event.user.name
  json.at      event.occurred_at
  json.comment event.comment
end
```

Tasks

- Generate and migrate ApprovalEvent
- Update SubmitService, ApproveService, RejectService to create events
- Show history in Jbuilder show template
- Test: submit → approve → check `/api/purchase_requests/:id` for history

Git Checkpoint

```
git commit -m "Day 27: ApprovalEvent model for audit trail"
```

Exercise — Do This On Your Own

Task: Add a `GET /api/purchase_requests/:id/history` endpoint that returns only the approval history for a PR in chronological order, including the time elapsed between each event (e.g. `"elapsed_from_previous": "2h 15m"`). For the first event, `elapsed_from_previous` should be `null`.

What you'll practice: dedicated history endpoint, time arithmetic, JSON formatting of computed fields

```
git commit -m "Day 27 exercise: Approval history endpoint with elapsed time"
```

Day 28 — Service Specs

Concepts

Service specs, `describe`, `context`, edge cases

```
RSpec.describe PurchaseRequests::SubmitService do
  let(:requester) { create(:user) }
  let(:pr)        { create(:purchase_request, user: requester) }

  describe "#perform" do
    context "when PR is draft and user is owner" do
      it "transitions to submitted" do
        described_class.new(pr, requester).perform
        expect(pr.reload.status).to eq("submitted")
      end

      it "sets submitted_at" do
        described_class.new(pr, requester).perform
        expect(pr.reload.submitted_at).to be_present
      end

      it "enqueues notification job" do
        expect {
          described_class.new(pr, requester).perform
        }.to have_enqueued_job(NotifyApproverJob)
      end
    end

    context "when PR is already submitted" do
      let(:pr) { create(:purchase_request, :submitted, user: requester) }

      it "returns false" do
        expect(described_class.new(pr, requester).perform).to be false
      end
    end

    context "when a different user tries to submit" do
      let(:other) { create(:user) }

      it "returns false" do
        expect(described_class.new(pr, other).perform).to be false
      end
    end
  end
end
```

Tasks

- Full spec for SubmitService (happy path + 3 failure cases)
- Full spec for ApproveService (approver can, requester cannot, guard check)
- Full spec for RejectService (with/without reason)
- Run `rspec spec/services/` — all pass

Git Checkpoint

```
git commit -m "Day 28: Service specs with edge case coverage"
```

Exercise — Do This On Your Own

Task: Write specs for `PurchaseRequests::BulkApproveService` (from Day 22 exercise). Cover: (1) all PRs succeed, (2) some PRs fail because they're in wrong state — verify the returned hash has correct `approved` and `failed` arrays, (3) verify that successfully approved PRs have `ApprovalEvent` records created, (4) failed PRs have no ApprovalEvent created.

What you'll practice: complex service specs, verifying side effects (DB records), partial success assertions

```
git commit -m "Day 28 exercise: BulkApproveService specs"
```

Day 29 — Final Polish & README

What to Build

Write a README a stranger could follow. Document every endpoint.

````
# Purchase Request Approval App

A Rails API for managing purchase requests and multi-level approval workflows.

## Stack
- Ruby 3.1 / Rails 7
- PostgreSQL
- Redis + Sidekiq (background jobs)
- Devise (authentication)
- AASM (state machine)

## Setup

```bash
git clone <repo>
cd purchase_approval_app
````
````

```
# Install dependencies
bundle install

# Start Postgres and Redis
brew services start postgresql@14
brew services start redis

# Database setup
rails db:create db:migrate db:seed

# Start Sidekiq (separate terminal)
bundle exec sidekiq

# Start server
rails s
```

API Endpoints

Method	Endpoint	Auth	Description
POST	/users/sign_in	No	Sign in, receive session cookie
GET	/api/purchase_requests	Yes	List all PRs (supports ?status=, ?page=, ?per_p
GET	/api/purchase_requests/:id	Yes	Get PR with approval history
POST	/api/purchase_requests	Yes	Create a new PR
POST	/api/purchase_requests/:id/submit	Yes	Submit draft PR
POST	/api/purchase_requests/:id/approve	Approver	Approve submitted PR
POST	/api/purchase_requests/:id/reject	Approver	Reject submitted PR
POST	/api/purchase_requests/bulk_approve	Approver	Approve multiple PRs
GET	/api/purchase_requests/summary	Yes	Count + total by status
GET	/api/purchase_requests/:id/history	Yes	Approval history with elapsed time
GET	/api/reports/top_requesters	Admin	Top requesters by spend
GET	/api/me	Yes	Current user info

Response Format

```json
{ "status": "success", "data": { ... } }
{ "status": "failure", "message": "Error description" }
```

Running Tests

```bash
rspec
```

Final Checklist

- [ ] `rspec` passes with zero failures

- [ ] No N+1 queries (check with bullet gem or logs)
- [ ] All controller actions delegate to services
- [ ] All state transitions go through AASM
- [ ] `config/approval_settings.yml` used in at least one validation
- [ ] README complete

Git Checkpoint

```
git add README.md
git commit -m "Day 29: README and API documentation"
git tag v1.0
git push origin main --tags
```

Exercise — Do This On Your Own

Task: Add the `bullet` gem (detects N+1 in development) to your Gemfile and configure it in `config/environments/development.rb`. Run `rails s`, hit the index endpoint, and check the logs for any bullet warnings. Fix any N+1 warnings that appear. Write a brief comment in the controller explaining why each `includes` call is needed.

What you'll practice: profiling tools, understanding N+1 in a real request, documentation habits

```
git commit -m "Day 29 exercise: Add bullet gem and fix any N+1 warnings"
```

Day 30 — Demo & Evaluation

Live Demo Flow

1. `rails db:reset db:seed` — fresh state
2. Sign in as Alice (requester)
3. Create a purchase request via curl/Postman
4. Submit it — show Sidekiq firing `NotifyApproverJob` in terminal
5. Sign in as Bob (approver), approve the PR
6. Fetch the PR — show approval history with timestamps
7. Try an invalid transition (approve a draft) — show the AASM error

Code Walkthrough — Be Ready to Explain

- Why does business logic live in services and not controllers?
- What does `before_action :set_purchase_request` save us from?

- What problem does `includes(:user)` solve?
- What does AASM give us that a plain `update(status:)` doesn't?
- Why does `after_commit` fire later than `after_save` ?
- Why does the backfill migration use `in_batches` instead of one big UPDATE?
- What is a concern and when should you use one vs a plain module?

Evaluator Checklist

- [ ] `rspec` passes with zero failures
- [ ] State machine rejects invalid transitions
- [ ] Approval history records correctly for all transitions
- [ ] Devise guards unauthenticated requests
- [ ] Controller actions are thin (no business logic inline)
- [ ] README is clear enough for a new engineer to set up the app

End of Zapro Engineering Bootcamp Guide Ruby Project: Library Management System — Days 1–10 Rails Project: Purchase Request Approval App — Days 11–30